

STUDENT GRADE & PERFORMANCE MANAGEMENT SYSTEM

Java Mini Project Report

Name: Prasaj Bajpai

Registration No.: 25BCE10864

Programme: B.Tech CSE

Academic Year: 2026–27

VIT Bhopal University

1. Introduction

The Student Grade & Performance Management System is a Java-based application designed to manage student academic information, marks, grades, and performance statistics.

The system provides a simple and structured way to add student records, store marks, calculate percentage and grades, search students, and generate overall class performance reports.

The project demonstrates the practical implementation of Object-Oriented Programming concepts, collections, input validation, modular programming, and basic report generation in Java.

2. Problem Statement

Managing student academic records manually can be time-consuming and may lead to calculation errors. Student details, marks, percentages, grades, and performance statistics need to be maintained in an organized manner.

The proposed Student Grade & Performance Management System provides a simple Java-based solution for managing student academic information and automatically generating academic results.

3. Objectives

- To create a simple student academic record management system.
- To store student details and subject-wise marks.
- To automatically calculate total marks and percentage.
- To automatically assign grades based on percentage.
- To determine student Pass/Fail status.

- To provide student search functionality.
 - To generate overall class performance statistics.
 - To demonstrate Java Object-Oriented Programming concepts.
-

4. Scope of the Project

The project focuses on basic student academic performance management.

The system allows users to:

- Add student details.
- Store subject marks.
- Calculate total marks.
- Calculate percentage.
- Generate grades automatically.
- Determine Pass/Fail status.
- Search for students.
- View all student records.
- Generate overall performance statistics.

The current version is a standalone Java application using in-memory data storage through ArrayList.

5. Functional Requirements

FR1 – Add Student

The system shall accept Student ID, student name, and course and create a new student record.

FR2 – View Students

The system shall display all student records currently stored in the application.

FR3 – Search Student

The system shall allow users to search for a student using Student ID.

FR4 – Enter Marks

The system shall accept marks for Mathematics, Java, DBMS, and DSA.

FR5 – Validate Marks

The system shall accept marks only between 0 and 100.

FR6 – Calculate Result

The system shall automatically calculate total marks and percentage.

FR7 – Grade Calculation

The system shall assign a grade according to the calculated percentage.

FR8 – Pass/Fail Evaluation

The system shall determine whether the student has passed or failed.

FR9 – Performance Report

The system shall calculate class average, highest percentage, lowest percentage, and pass/fail statistics.

6. Non-Functional Requirements**6.1 Performance**

The system should process student records and calculate results quickly for normal classroom-sized datasets.

6.2 Usability

The application should provide a simple menu-driven interface that can be easily understood by users.

6.3 Reliability

The system should provide consistent calculations for total marks, percentage, grades, and results.

6.4 Maintainability

The application is divided into separate classes so that individual components can be modified easily.

6.5 Error Handling

The system validates marks and prevents invalid values outside the range of 0 to 100.

6.6 Scalability

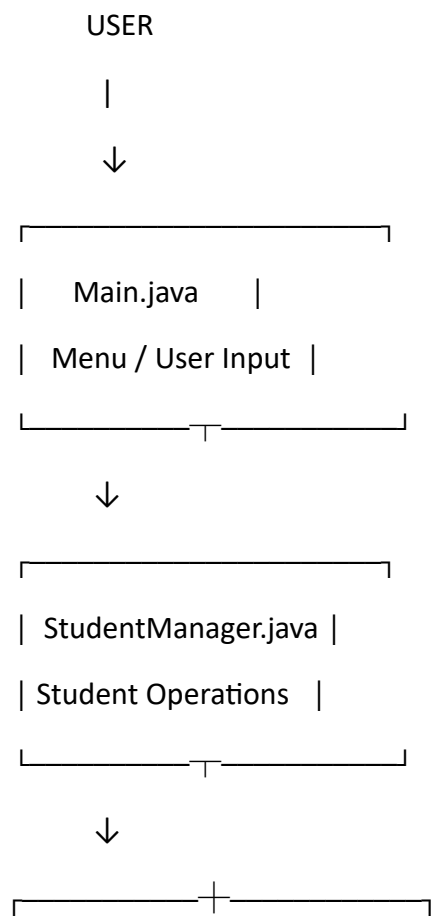
The system uses an ArrayList structure that allows multiple student records to be maintained during program execution.

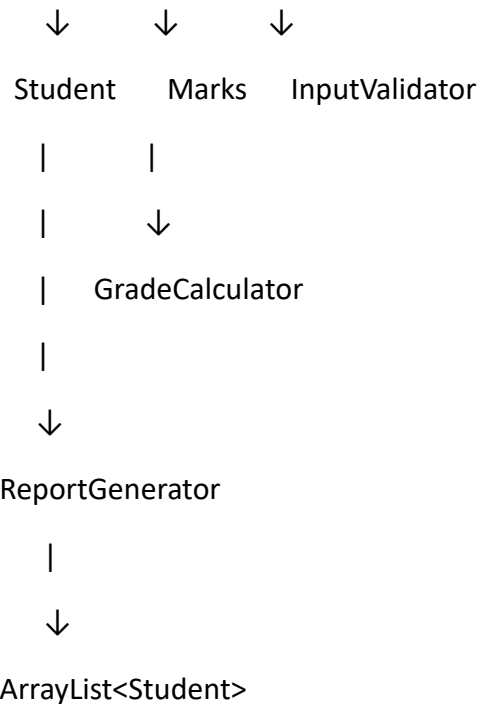
7. Technologies Used

Technology	Purpose
Java	Core programming language
Object-Oriented Programming	Application design
ArrayList	Student record storage
IntelliJ IDEA	Development environment
Java Scanner	User input
Git/GitHub	Version control and repository

8. System Architecture

The project follows a simple modular architecture.

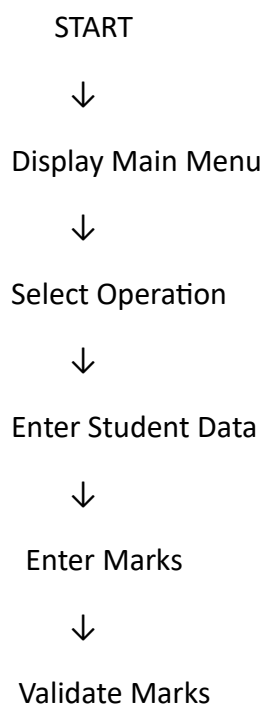


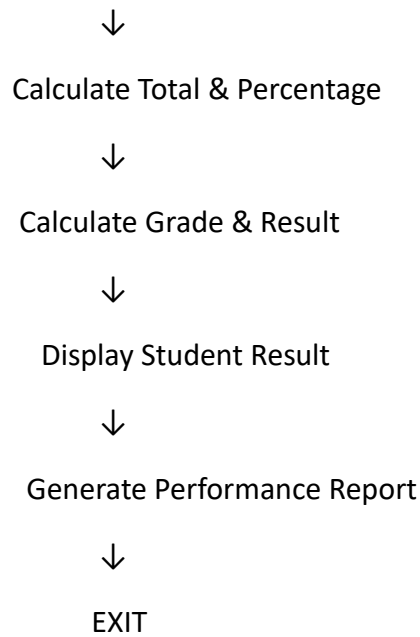


Architecture Explanation:

The Main class controls the user interaction and menu. StudentManager manages student-related operations. Student stores student information, while Marks handles academic marks and calculations. GradeCalculator calculates grades and Pass/Fail status. InputValidator validates user input. ReportGenerator produces overall performance statistics.

10. Workflow Diagram



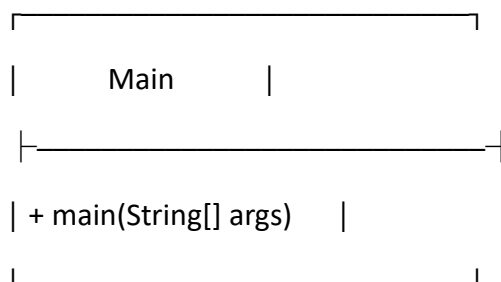


Explanation

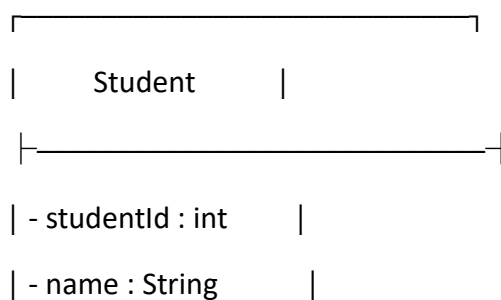
The system starts by displaying the main menu. The user selects an operation and provides the required information. When marks are entered, they are validated before total marks and percentage are calculated. The system then determines the grade and Pass/Fail status and displays the result.

11. Class Diagram

Main



Student



```

| - course : String      |
| - marks : Marks       |
|_____|
| + getId()             |
| + getName()           |
| + getCourse()         |
| + getMarks()          |
| + setMarks()          |
| + displayStudent()    |
|_____|

```

Marks

```

|_____|
|      Marks           |
|_____|
| - mathematics : double |
| - java : double       |
| - dbms : double       |
| - dsa : double        |
|_____|
| + getTotal() : double  |
| + getPercentage() : double |
| + displayMarks()      |
|_____|

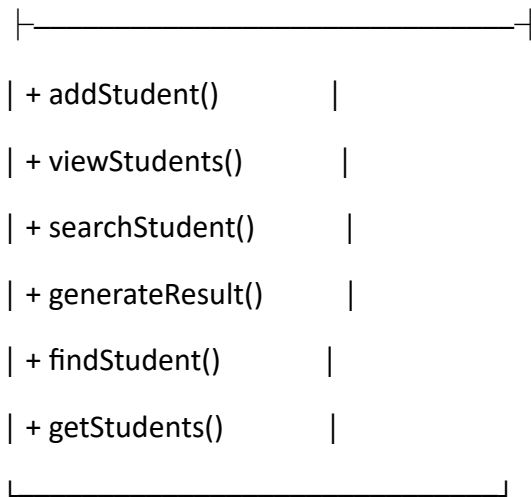
```

StudentManager

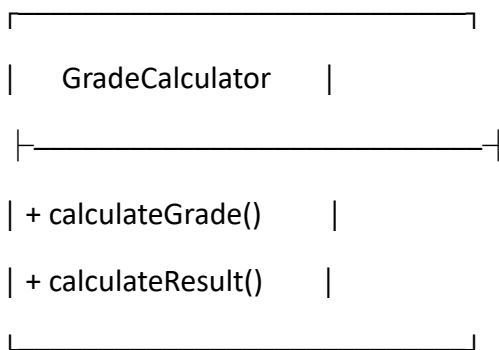
```

|_____|
|      StudentManager  |
|_____|
| - students : ArrayList |

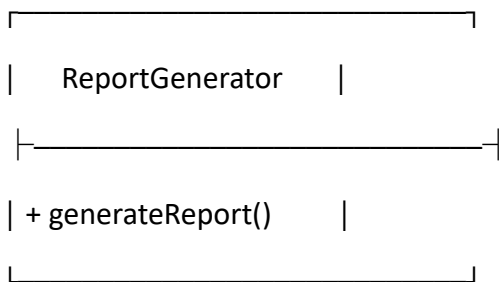
```



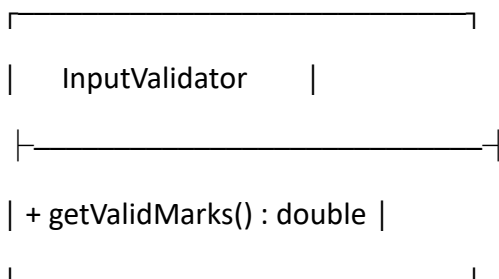
GradeCalculator



ReportGenerator



InputValidator



Relationships

Main



StudentManager



Student —→ Marks



GradeCalculator



ReportGenerator

StudentManager —→ InputValidator

12. Sequence Diagram

Scenario: Generate Student Result

User



| Select Generate Result



Main



| generateResult()



StudentManager



| findStudent(ID)



Student



| getMarks()



Marks



| getPercentage()



StudentManager



| calculateGrade()



GradeCalculator



| Return Grade



StudentManager



| calculateResult()



GradeCalculator



| Return PASS/FAIL



StudentManager



| Display Result



User

Explanation

When the user selects the Generate Result option, the request is handled by StudentManager. The system searches for the student and retrieves the marks. The Marks class calculates the percentage, while GradeCalculator determines the grade and Pass/Fail status. The final result is displayed to the user.

13. Database / Storage Design

The current version of the project does not use an external database.

Student records are temporarily maintained using:

`ArrayList<Student>`

Therefore, an ER diagram is **not applicable to the current version**.

Database integration using JDBC and MySQL is planned as a future enhancement.

14. Design Decisions & Rationale

Java

Java was selected because the project is intended to demonstrate Java programming and Object-Oriented Programming concepts.

ArrayList

ArrayList was selected because it provides simple dynamic storage for multiple student records without requiring database configuration.

Modular Classes

Different classes were created for student management, marks, grading, validation, and reporting to improve maintainability.

GradeCalculator

Grade calculation was separated into its own class so that grading rules can be modified independently.

InputValidator

Input validation was separated into a dedicated class to prevent invalid marks.

ReportGenerator

Report generation was separated from student management to maintain a clear division of responsibilities.

15. Implementation Details

The application is implemented using seven Java classes:

Main.java

Contains:

```
public static void main(String[] args)
```

and controls the menu-driven application.

Student.java

Stores Student ID, name, course, and marks.

Marks.java

Stores subject-wise marks and calculates total marks and percentage.

GradeCalculator.java

Calculates grade and Pass/Fail status.

StudentManager.java

Manages the student collection and performs add, view, search, and result operations.

ReportGenerator.java

Calculates class average, highest percentage, lowest percentage, and pass/fail counts.

InputValidator.java

Validates marks between 0 and 100.

16. Grade Calculation

The system uses the following grading scheme:

Percentage	Grade
90% and above	A+
80–89.99%	A
70–79.99%	B
60–69.99%	C

Percentage	Grade
------------	-------

50–59.99%	D
-----------	---

Below 50%	F
-----------	---

The Pass/Fail criterion used in the application is:

40% and above → PASS

Below 40% → FAIL

17. Testing Approach

The project was tested using different functional test cases.

Test Case Input/Action		Expected Result	Status
TC01	Add valid student	Student added successfully	Pass
TC02	Marks above 100	Invalid marks message	Pass
TC03	Search existing ID	Student details displayed	Pass
TC04	Search invalid ID	Student not found	Pass
TC05	Generate result	Total, percentage, grade displayed	Pass
TC06	Generate performance report	Statistics displayed	Pass

19. Sample Result

For testing purposes, the following marks were entered:

Subject	Marks
Mathematics	85
Java	90
DBMS	78
DSA	88
Total	341/400
Percentage	85.25%

Subject	Marks
----------------	--------------

Grade	A
--------------	----------

Result	PASS
---------------	-------------

The system automatically calculates the total, percentage, grade, and Pass/Fail status.

20. Challenges Faced

- Dividing the project into multiple Java classes.
 - Managing student records using ArrayList.
 - Validating marks between 0 and 100.
 - Connecting marks with percentage and grade calculations.
 - Handling student search functionality.
 - Calculating class-level performance statistics.
 - Maintaining a simple and understandable program structure.
-

21. Learnings & Key Takeaways

Through this project, the following concepts were learned and applied:

- Classes and objects.
- Encapsulation.
- Constructors.
- Methods.
- Static methods.
- ArrayList.
- Conditional statements.
- Loops.
- Input validation.
- Modular programming.
- Basic error handling.

- Result and statistical calculations.
 - Software documentation and testing.
-

22. Future Enhancements

The project can be enhanced in the future by adding:

1. JDBC and MySQL database integration.
 2. Java Swing/JavaFX graphical interface.
 3. User authentication.
 4. Student and faculty login.
 5. Attendance management.
 6. Subject-wise performance charts.
 7. PDF report generation.
 8. Excel/CSV export.
 9. Persistent student records.
 10. Cloud-based storage.
-

23. Conclusion

The Student Grade & Performance Management System provides a simple Java-based solution for managing student academic performance.

The system automates basic academic calculations, reduces manual effort, provides student search and management functionality, and generates useful performance statistics.

The project demonstrates the practical application of Java programming concepts and Object-Oriented Programming through a modular and maintainable implementation.

The project also provides a foundation for future development through database integration, graphical interfaces, authentication, and advanced analytics.

24. References

1. Oracle Java Documentation.
2. Java Platform Standard Edition Documentation.

3. Java Object-Oriented Programming concepts.
4. Course notes and learning material provided through VITyarthi.
5. Java Collections Framework documentation.